❷ Storing the lambda expression as a variable means you can name it—getFruit in this case.

❸ Handlers can be static methods in any class.

❹ Handlers can also be instance methods.

❺ You can also use local functions, introduced in C# 7.0, as handler methods.

❻ Handlers can also be instance methods.

❼ Converts the response to a JsonObject

Listing 5.2 demonstrates the various ways you can pass handlers to an endpoint by simulating a simple API for interacting with a collection of Fruit items:

- A lambda expression, as in the `MapGet("/fruit")` endpoint
- A `Func<T, TResult>` variable, as in the `MapGet("/fruit/{id}")` endpoint
- A static method, as in the `MapPost` endpoint
- A method on an instance variable, as in the `MapPut` endpoint
- A local function, as in the `MapDelete` endpoint

All these approaches are functionally identical, so you can use whichever pattern works best for you.

Each `Fruit` record in listing 5.2 has a `Name` and a `Stock` level and is stored in a dictionary with an `id`. You call the API by using different HTTP verbs to perform the CRUD operations against the dictionary.

WARNING This API is simple. It isn't thread-safe, doesn't validate user input, and doesn't handle edge cases. We'll remedy some of those deficiencies in section 5.3.